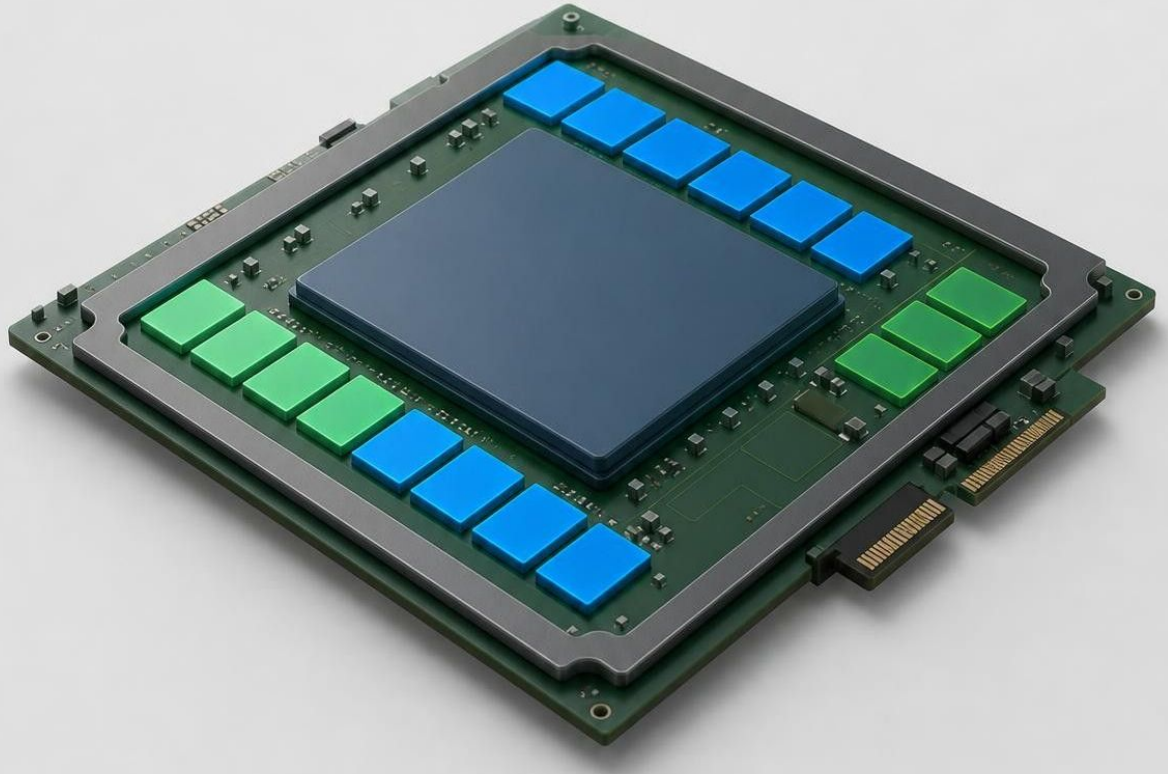
Will this LLM fit on my GPU?

How to calculate the GPU memory an LLM needs, explained from scratch

The first question is not "how smart?" or "how fast?" but **"will it fit in GPU memory?"** If not, nothing else matters.

By the end: "a 13B model on a 24 GB GPU" → a yes or no in under a minute, and what to change.

VRAM WEIGHTS KV CACHE



What we will cover

01

The basics

Why memory comes first, what VRAM is, what sits in GPU memory, what "7B" or "70B" means

02

Weights

Bytes per weight and precision, the weight-memory formula, quantization and what we lose

03

The KV cache

What it is, how big it gets with long text and many users, how Grouped-Query Attention shrinks it

04

Training

Why it needs about 8× more memory, how LoRA and QLoRA make it fit on one GPU

05

Case studies

Real case studies and a final cheat sheet

GPU memory is the first question to ask

- A GPU only does math on numbers already in its own memory, so the whole model must be copied in first
- Model bigger than memory? "Out of memory" error: it doesn't run slowly or partly, it **fails**
- Only two ways out: make the model take less memory, or get more memory (bigger GPU or more GPUs)
- This calculation decides which GPU to buy or rent, the monthly cloud bill, and whether we must shrink the model

An 80 GB cloud GPU costs many times more per hour than a 16 GB one, so fitting the smaller GPU directly decides the budget.



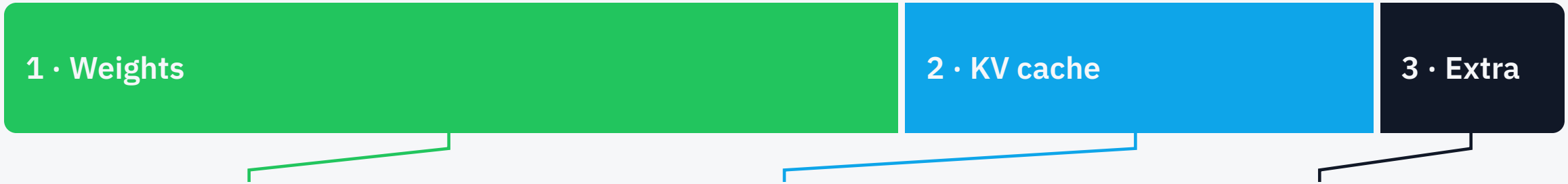


VRAM: why the GPU needs its own memory

- A computer has main memory (RAM); a GPU has its own, VRAM (Video RAM), named from its graphics days
- Reason: speed. VRAM sits right next to the chip and feeds thousands of cores extremely fast
- Path: normal RAM → PCIe connection (slower) → VRAM → GPU cores (extremely fast) → math
- Data-centre GPUs read VRAM at a few terabytes per second; PCIe is many times slower
- Fetching weights across PCIe is called **offloading**: it works, but runs many times slower

For practical speed, the whole model should fit inside VRAM.

What sits in GPU memory when we run a model



Weights (usually the biggest): what the model learned; fixed size, unchanged while running

KV cache (grows with text length and users): saved information about earlier tokens so they aren't recalculated

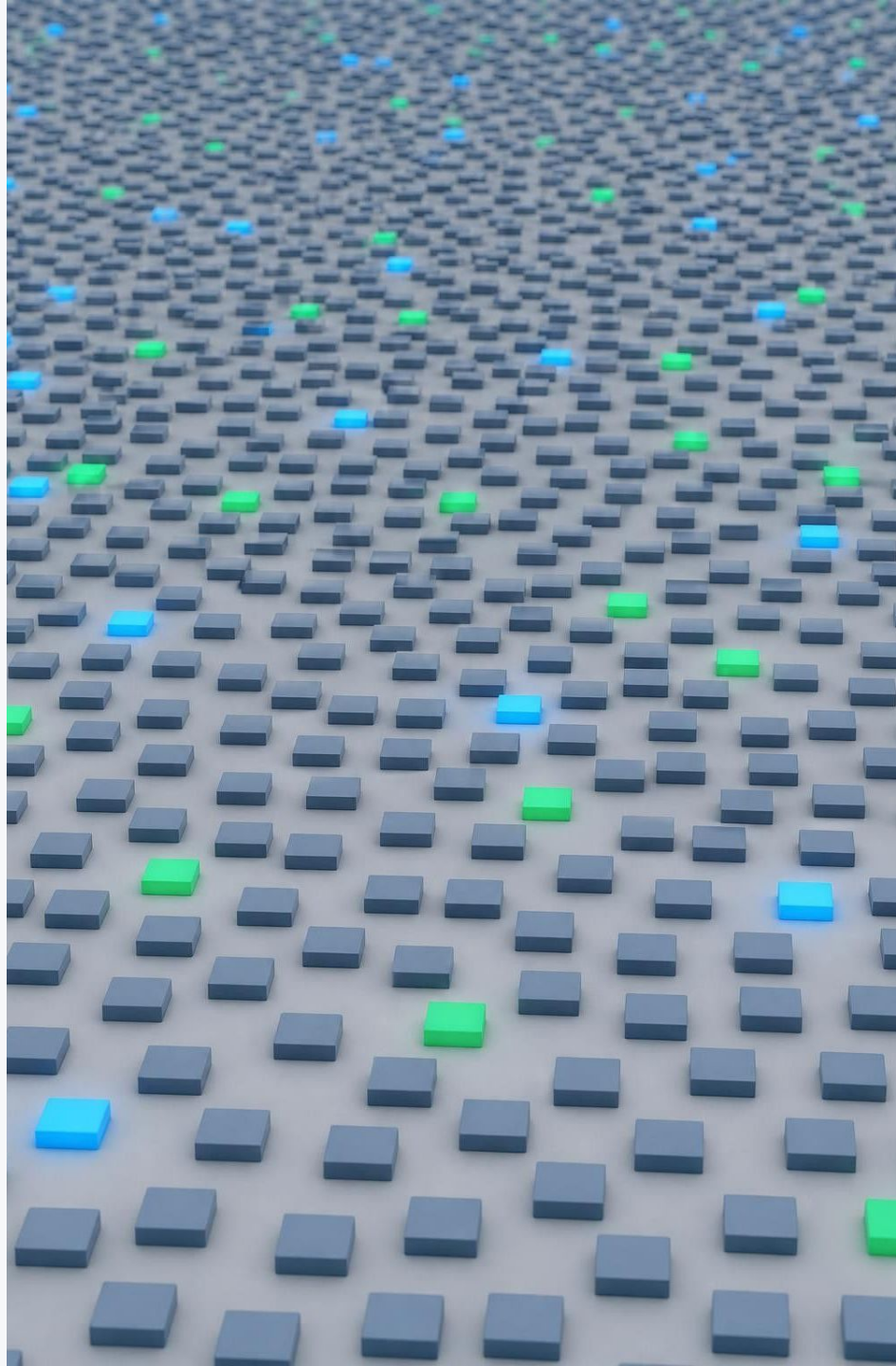
Activations + overhead (small extra): in-between numbers from each layer, plus memory used by PyTorch and CUDA

Running a model is called inference. Training puts even more in memory (Part 4). We start with the biggest part: weights.

What ”7B” or ”70B” really means

- B = billion parameters; a parameter (or weight) is one number learned during training
- 7B ≈ 7,000,000,000 numbers; 70B ≈ 70,000,000,000; GPT-3 (OpenAI, 2020) had 175 billion
- Every one must sit in GPU memory: every token passes through every layer, and every layer uses its weights

⚠ **Very important:** parameter count is size, not smartness. A well-trained 8B model from 2024 can beat a poorly trained 30B from 2022, but for memory, more parameters always means more memory.



How many bytes does each weight take?

Precision = how precisely we store each number. A byte is the basic unit of memory. FP = floating point (decimals); INT = integer, scaled back into decimals.

Precision	Bytes per weight	How precise	Where used
FP32	4	About 7 correct digits	Older training, very precise work
FP16	2	About 3 correct digits	Standard for running models
BF16	2	2–3 digits, handles big ranges	Standard for training modern models
INT8	1	Only 256 possible values	Quantized models
INT4	0.5	Only 16 possible values	Heavily quantized models

Weight 0.0372918: FP32 ≈ 0.0372918 · FP16 ≈ 0.03729 · INT8 ≈ 0.0373 · INT4 ≈ 0.04

The formula: weights × bytes per weight

Weight memory = Number of weights × Bytes per weight

- 7B model: FP32 = 28 GB · FP16 = 14 GB · INT8 = 7 GB · INT4 = 3.5 GB
- 1 billion bytes ≈ 1 GB, so billions × bytes per weight = GB directly
- Shortcuts: per 1 billion weights, FP16 ≈ 2 GB, INT8 ≈ 1 GB, INT4 ≈ 0.5 GB

13B IN FP16

26 GB

13 × 2 = 26 GB for weights alone, too big for a 24 GB GPU before anything else is counted

Weight memory for common model sizes

Model size	FP16	INT8	INT4
1B	2 GB	1 GB	0.5 GB
7B / 8B	14–16 GB	7–8 GB	3.5–4 GB
13B	26 GB	13 GB	6.5 GB
70B	140 GB	70 GB	35 GB
175B (GPT-3 size)	350 GB	175 GB	88 GB
405B	810 GB	405 GB	203 GB

- Memory grows in a straight line: 70B needs exactly 10× a 7B
- FP16 → INT4 cuts memory to a quarter, the biggest single saving
- The biggest models fit on no single GPU (80–192 GB max): 175B in FP16 needs at least five 80 GB GPUs, more with KV cache. This is one reason GPT-3-size models are offered through APIs

Quantization: smaller weights, a little rounding

Storing each weight with fewer bytes than it was trained with (e.g. BF16 → INT8 or INT4).

1. Take a group of weights, say 64
2. Find the smallest and largest value
3. Divide the range into steps: 256 for INT8, 16 for INT4
4. Replace each weight with its nearest step
5. Save one small scale number per group to turn steps back into decimals

INT8: Answers almost the same; most can't tell

INT4: Slightly worse on hard reasoning, math, rare facts; fine for chat and summaries

Below 4 bits: Quality drops clearly

A 13B in INT4 (~6.5 GB) often beats a 3B in FP16 (~6 GB), so quantization is usually tried first.

Llama 3 8B on a free Colab GPU

A student wants Meta's Llama 3 8B on Colab's free T4 with **16 GB**.

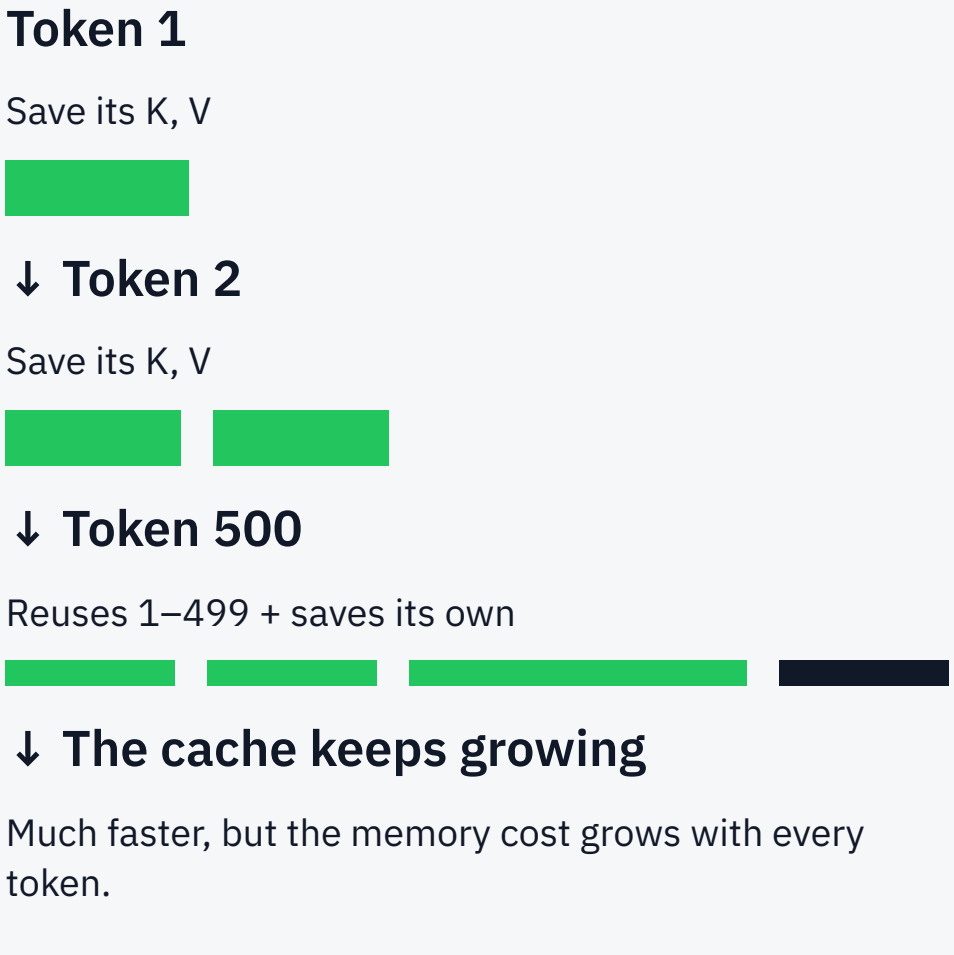
Precision	Weights	Result
FP16	8B × 2 = 16 GB	The whole GPU, no room for KV cache or software → **out of memory**
INT8	8B × 1 = 8 GB	Leaves ~8 GB → **works**
INT4	8B × 0.5 = 4 GB	Leaves ~12 GB → **works comfortably**, room for longer chats

Lesson: the same model fails or succeeds on the same GPU depending only on precision. That's why model sites offer "Q4" or "4-bit" versions, and why the free T4 is enough for learning with 8B models.



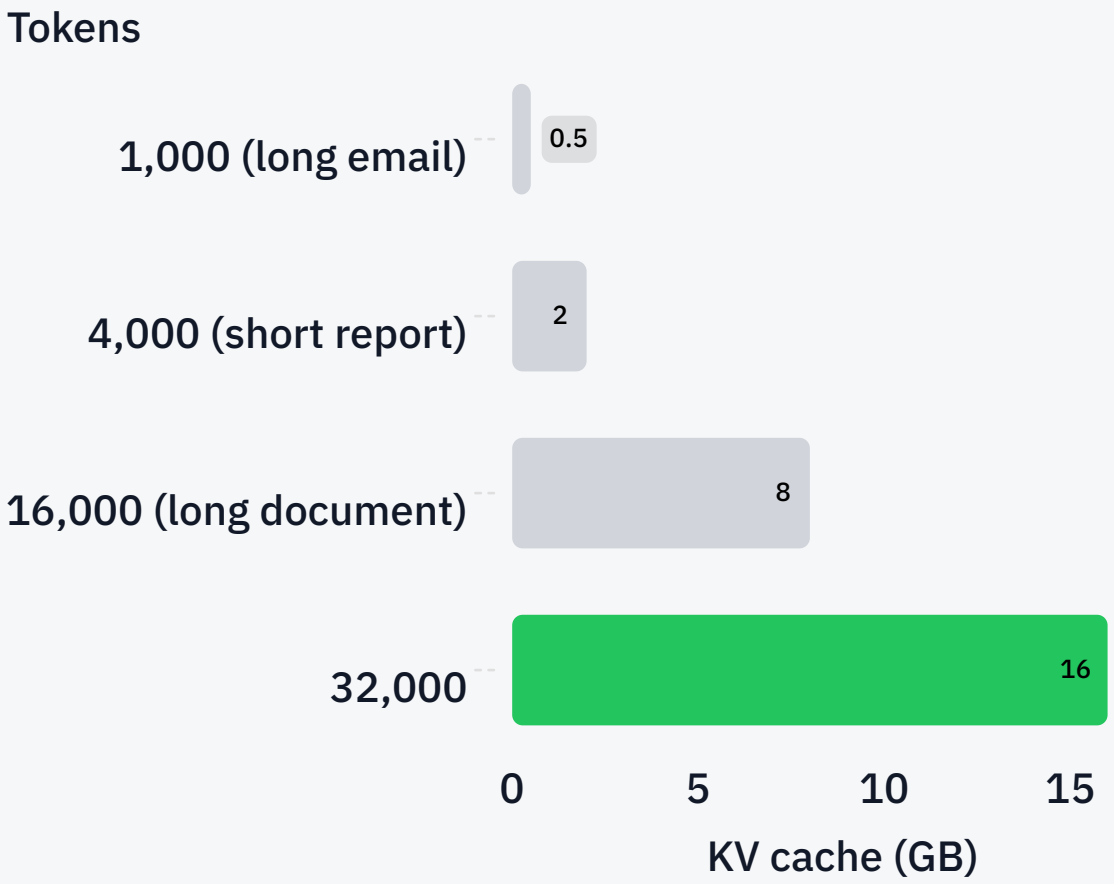
The KV cache: saved work so tokens aren’t recalculated

- An LLM writes one token (a word or part of a word) at a time
- To pick the next token, attention looks back at every earlier token using two lists of numbers: the Key and the Value
- Without saving, token 501 would recalculate K and V for all 500 earlier tokens, then 502 for all 501: huge repeated work
- So each token's K and V are calculated once, saved in GPU memory, and reused: that's the **KV cache**



How big the KV cache gets

KV cache = 2 × layers × size of K or V per layer × bytes per number × tokens (the 2 is for Key and Value)



Llama 2 7B: 32 layers, 4,096 numbers each, FP16 → 2 × 32 × 4,096 × 2 bytes ≈

0.5 MB

per token

At 32,000 tokens the KV cache (16 GB) is bigger than the 7B model's own FP16 weights (14 GB). For long text, it can become the biggest user of GPU memory.

Grouped-Query Attention: 4× smaller KV cache

Llama 2 7B · 32 heads → 32 K/V sets



Llama 3 8B · 32 heads share 8 K/V sets



Blue = heads · Green = saved K and V sets (half of each model shown)

- In GQA, several heads share one K and V
- Quality stays very close, so almost every major open model since 2023 uses GQA or similar

Point	Llama 2 7B	Llama 3 8B (GQA)
Layers	32	32
Sets of K and V saved	32	**8**
KV cache per token (FP16)	~0.5 MB	**~0.13 MB**
KV cache for 32,000 tokens	~16 GB	**~4 GB**

Llama 3 8B is slightly bigger, yet its cache is four times smaller.

Lesson: don't just compare parameter counts; check for GQA.



A long-document legal assistant

A law firm reads full contracts, often ~**100,000 tokens**, with Llama 3 8B (long context + GQA).

WEIGHTS FP16

16 GB

8B × 2

KV CACHE, 1 USER

~13 GB

~0.13 MB × 100,000

TOTAL

29 GB

before overhead

A 24 GB GPU isn't enough, even though the weights alone fit.

Options: INT8 weights (8 + 13 = 21 GB, fits 24 GB) · 8-bit KV cache (~6.5 GB) · a 48 or 80 GB GPU (easy, but costs more per hour)

Lesson: for long documents, text length matters as much as model size.

A chatbot crashes on launch day

A startup's support bot on Llama 2 7B works on a 24 GB GPU in testing, then crashes when 30 customers chat at once.

	1 user	30 users
Weights (FP16, shared)	14 GB	14 GB
KV cache (2,000 tokens each)	~1 GB	30 × 1 = 30 GB
Total	~15 GB → fits	**~44 GB → doesn't fit**

Key lesson: weights are shared; the KV cache is per user.

Fixes: limit simultaneous users with a short queue · cap conversations at 2,000 tokens · switch to a GQA model (~4× smaller cache) · quantize weights to free room



44 GB

needed for 30 users on a 24 GB GPU

Activations, overhead, and the full formula

Activations

In-between results passed layer to layer; small and temporary during inference

Overhead

CUDA reserves memory at start, PyTorch and serving tools keep bookkeeping memory, and small unusable gaps appear

TOTAL = WEIGHTS + KV CACHE + ACTIVATIONS AND OVERHEAD

Total ≈ (Weights + KV cache) × 1.2

The extra 20% is a planning margin. If you're near the limit, assume it won't fit.

Example: 7B in INT8 (7 GB) + 4,000-token chat (2 GB) → (7 + 2) × 1.2 ≈ **11 GB**, comfortable on 16 GB.



Running Llama 3 70B privately

A company can't send customer data to an outside API, so it runs Llama 3 70B on its own H100s (**80 GB** each).

Option	Weights	Fit
FP16	140 GB	Not on one H100; two leave ~20 GB for a few short chats; four for many users
INT8	70 GB	One H100 leaves ~10 GB, very tight; two have plenty of room
INT4	35 GB	One H100 leaves ~45 GB; with GQA (~0.33 MB/token) holds well over 100,000 tokens across users

Decision: quality first (legal, medical) → FP16 on several GPUs; cost first → INT4 on one GPU. **Lesson:** one GPU or four, depending on precision and users, can change the monthly bill several times.

Mixture of Experts needs the full model's memory

- In MoE, each layer has several smaller sub-networks (experts); a router picks a few per token
- **Mixtral 8x7B** (Mistral AI, Dec 2023): 8 experts per layer, 2 used per token; ~46.7B total, ~12.9B used per token
- Myth: "it needs memory like a 13B model." Wrong: the next token may need any expert, so all must stay in memory

Point	What people expect	The real answer
Parameters used per token	~13B	~13B
Parameters in GPU memory	~13B	**~47B**
Weight memory in FP16	~26 GB	**~93 GB**

MoE gives **speed**, not memory savings. Always use the total parameter count.

Training needs about 16 bytes per weight

What we store	Bytes	Why
The weight itself (BF16)	2	Used to make guesses
Its gradient (BF16)	2	How much and which way to change it
Precise copy (FP32)	4	Tiny changes would get lost in 2-byte rounding
Adam value 1	4	Running average of recent gradients
Adam value 2	4	Running average of gradient size
Total	**16**	

FULL TRAINING

16 bytes

per weight, vs ≈ 2 bytes for running: about 8× more, before activations

Each step changes a weight by ~0.0000001; BF16 would round it away, so an FP32 copy lets changes add up.

Adam, the most common optimizer, keeps two running averages per weight for smoother learning: 8 extra bytes.

Activations during training, and gradient checkpointing

1

Forward pass

Makes a guess; every layer's activations stay in memory

2

Measure loss

How wrong the guess was

3

Backward pass

Uses those results to find how each weight should change

- Size depends on batch size, sequence length (tokens per example), and model size (layers and width)
- With long examples and big batches, activations can reach tens of GB, sometimes more than the weights

Gradient checkpointing: save only some layers' activations (say every fourth) and recalculate the rest in the backward pass. Gain: much less activation memory. Cost: training ~20–30% slower. Used in almost every large training job.

Fine-tuning a 7B model on one 24 GB GPU

A small team wants a 7B model to learn their product and tone from support tickets.

FULL FINE-TUNING

112 GB

QLORA

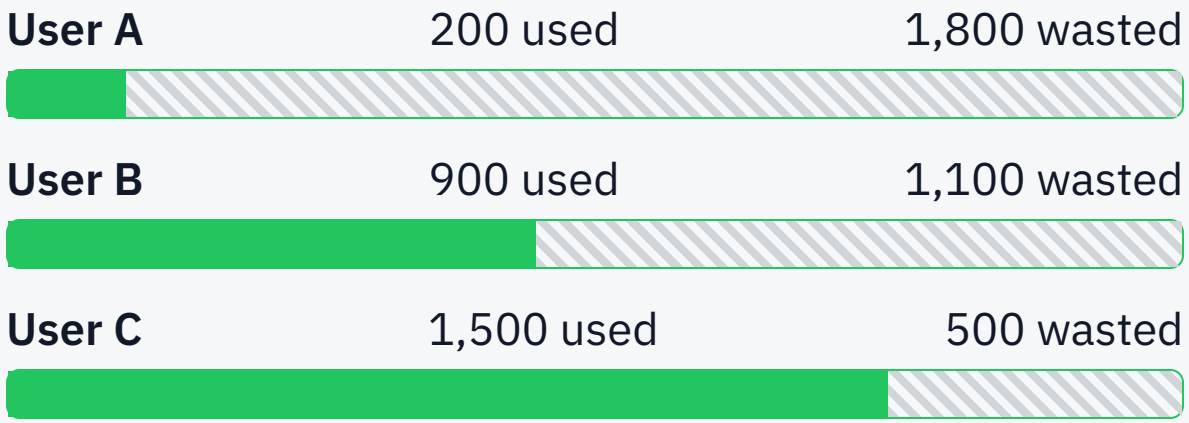
6–10 GB

Option	Memory	Fits 24 GB?
Full fine-tuning	7B × 16 = 112 GB + activations (≥ two 80 GB GPUs)	No
LoRA	14 GB frozen + <1 GB trainable + activations ≈ 16–20 GB	Yes
QLoRA	3.5 GB frozen (4-bit) + tables + activations ≈ 6–10 GB	Yes, even a free 16 GB T4

LoRA freezes original weights (2 bytes each, no gradients or optimizer values) and trains small added tables, under 1% of the model. **QLoRA** is LoRA with frozen weights in 4-bit; the 2023 paper fine-tuned a 65B model on a single 48 GB GPU, close to full 16-bit results.

Lesson: most companies don't need full fine-tuning.

vLLM stops wasted serving memory



Each bar = 2,000 tokens reserved · green used · hatched wasted

Older servers reserved KV cache for the maximum length (e.g. 2,000 tokens) per user, whatever they needed. Reserved but never filled, and no one else could use it.

Fix: in 2023, UC Berkeley released vLLM with **PagedAttention**: memory handed out in small pieces as the conversation grows, returned to the pool when a user finishes.

Result: waste fell from often more than half to a few percent, so the same GPU served many more users.

Lesson: a good serving tool can be worth as much as a bigger GPU.

Five ways to make a model fit

Way	What it shrinks	What we give up
1. Quantize the weights	Weights	A little accuracy, especially at 4-bit
2. Shorten the context	KV cache	The model sees less text at once
3. Serve fewer users together	KV cache	Some users wait a little longer
4. Choose a model with GQA	KV cache	Almost nothing
5. Split across many GPUs	Nothing, it adds memory	More cost, setup, some speed

For training, add: LoRA or QLoRA · gradient checkpointing · smaller batches

Often forgotten: pick a smaller model; a well-trained 8B handles many business tasks, so test it first

Quantization is usually tried first: biggest saving for smallest quality loss

Cheat sheet: the whole deck on one slide

- **Bytes per weight:** FP32 = 4 · FP16/BF16 = 2 · INT8 = 1 · INT4 = 0.5
- **Weight memory:** billions of parameters × bytes per weight = GB
- **KV cache:** grows with tokens and users; check for GQA
- **Running:** Total ≈ (Weights + KV cache) × 1.2 · **Full training:** ~16 bytes per weight + activations

GPU	Memory	Comfortably runs (inference)
Colab T4	16 GB	7B / 8B in INT8 or INT4
RTX 4090	24 GB	7B in FP16, 13B in INT8
A100 / H100	80 GB	70B in INT4, 13B in FP16 with many users

ASK EVERY TIME

1. Parameters, counting all experts?
2. Precision?
3. Text length, and GQA?
4. Users at once?

Multiply, add 20%, compare with GPU memory.

Too big? Quantize first.